CO4015 MComp Project

Interim Report

“Tracer Racers” – An Implementation of the “Racetrack” Game

Muhammad Mogra

20th November 2025

School of Computing and Mathematical Sciences
University of Leicester

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s).

Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s).

I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Muhammad Mogra

Signed: 

Table of Contents

1. Introduction 5
   1.1. Background 5
   1.2. Project Aims 5
   1.3. Project Objectives 5
2. Existing Games 6
   2.1. Main Reference Points 6
   2.2. Primary Objectives Relations 6
      2.2.1. Primary Objective 4.1 7
      2.2.2. Primary Objective 4.2 8
      2.2.3. Primary Objective 4.3 9
      2.2.4. Primary Objective 4.4 9
      2.2.5. Primary Objective 4.5 10
   2.3. Secondary Objectives Relations 10
      2.3.1. Secondary Objective 1 10
      2.3.2. Secondary Objective 2 11
      2.3.3. Secondary Objective 3 11
   2.4. Other Games 12
3. Development Tools and Frameworks 12
   3.1. Development Areas 12
   3.2. Technology Decisions 13
      3.2.1. Front End and Rendering 13
      3.2.2. Back End 13
      3.2.3. Databases and Firebase 13
      3.2.4. Multiplayer Networking 13
      3.2.5. Game Engines 13
      3.2.6. Further Considerations 14
4. Application Requirements 14
   4.1. Requirements Glossary 14
   4.2. Requirements List 15
5. Implementation & Design 18
   5.1. Client-Server Architecture 18
   5.2. Screen Navigation 19
   5.3. Track Building 19

5.3.1. Racetrack Representation 20
5.3.2. Track Builder Interface 21
5.4. Track Listing, Selection, and Management 21
5.5. Game Creation and Race Options 22
5.6. Race Gameplay 23
5.6.1. Move Selection 23
5.6.2. AI Pathfinding 23
5.7. Car Customisation 24
5.8. App Settings 25
5.9. Game Handling and Data 25
5.9.1. Documents and Objects 25
5.9.2. Holistic Class Diagram 26
6. Application Testing 27
7. Project Timeline 27
7.1. Current Progress 27
7.2. Workload and Future Timeline 28
7.3. Future Timeline Gantt Charts 30
7.4. External Deadlines 31
8. References 31

1. Introduction

1.1. Background

The *Racetrack* game[1] is a turn-based pencil-and-paper (i.e., 2D) game where players must race around a track using vectors on a grid, as opposed to continuous 3D space. Players can select a cell to move to within their acceleration limit, typically within 1 unit, where the eight neighbours they can select are determined by their current velocity. Selecting a tile then determines the player's subsequent velocity and the cycle repeats. Players must monitor their velocity and plan to make their way around without crashing.

The Interim Report outlines the project's aims and objectives, as well as research and discussions on approaches to achieve them in the form of requirements. There is also an overview of the architecture and initial designs.

1.2. Project Aims

A few computer games based on the *Racetrack* concept exist but tend to be simple with minimal gameplay features, or each one has features that others do not that would be interesting to combine. This project aims to recreate and build upon the *Racetrack* game in these areas.

The project is primarily planned to include track building capabilities, the ability to play tracks and compete against other players locally, and AI to race against, formed as a single cohesive application. These features at least provide an experience for players to challenge themselves on custom made tracks and against other drivers for an engaging experience[11]. While the concept of vectors is mathematical, the project is focused more on gameplay than education, but players may use it so[2].

If time permits, further features may be implemented such as online multiplayer capabilities and customisation to enhance the experience and connect with other players, beyond just locally[3]. It may also include varying AI difficulties to ensure they are not too perfect to compete against.

1.3. Project Objectives

Based on the aims described above, the objectives for the project can be split into two sections: the **Primary Objectives**, which must be completed to ensure the project is successful and fulfils the aims; and **Secondary Objectives**, which cover more challenging areas that enhance gameplay, though are not necessary for the project's success.

<u>Primary Objectives</u>

1. Compile game requirements based on the aims
2. Design the game architecture
3. Draw screen and gameplay designs
4. Implement the functionality for gameplay features
    1. Enable players to race around tracks with vectors
    2. Enable players to create tracks
    3. Enable players to manage their tracks
    4. Enable players to race against other players locally
    5. Enable players to race against AI racers
5. Perform tests to ensure all functionality operates as expected

Secondary Objectives

1. Enable players to race against other players online
2. Enable players to race against AI racers of varying difficulty
3. Enable players to personalise their gameplay experience

While the **Secondary Objectives** are optional, it is still important to factor them in for all subsequent research and design decisions, as knowledge and planning for them will their inclusion much easier if time permits[4]. As such, they have been considered throughout the Interim Report.

2. Existing Games

When developing a new application, it is useful to research existing ones in the same area to understand how results have been achieved[5]. In the project's case, this would include how the Racetrack concept has been implemented and any gameplay elements. This provides ideas for features which would be interesting, helpful, or challenging to ensure a positive gameplay experience[11] and fulfilling project. Potential improvements can be considered for the project's version, as there may be elements some or none of the apps include.

2.1. Main Reference Points

The Google search queries "racetrack game" and "racetrack game vector" were used to find games to research. While there were a fair number of games, the following were chosen as they are either most like the *Racetrack* concept or contribute significantly to another objective:

1. **Inertia Racetrack[6]:** a simple web-app recreation of the *Racetrack* game, allowing two players to compete locally across ten unique tracks. Players take turns to drive their race cars around the track until a player reaches the finish line.
2. **Vector Racer[7]:** another web-based recreation of the *Racetrack* game with similar gameplay to *Inertia Racetrack,* allowing players to compete with a mix of up to six local players or AI drivers with varying difficulties. In addition to gameplay, it introduces track building capabilities with materials and ranked leaderboards.
3. **PolyTrack[8]:** a 3D, low-polygon racing game which plays much like most racing games where racers can drive freely rather than be bound to a grid. Though it does not incorporate the *Racetrack* concept, it has a sophisticated track building system with diverse parts, leaderboards and track replays, and player personalisation options

2.2. Primary Objectives Relations

This section compares each game based on the subobjectives of **Primary Objective 4**.

(section continues)

2.2.1. Primary Objective 4.1

"Enable players to race around tracks with vectors"

Inertia Racetrack and *Vector Racer* both conform to the *Racetrack* concept's rules, defined in **Section 1.1**.

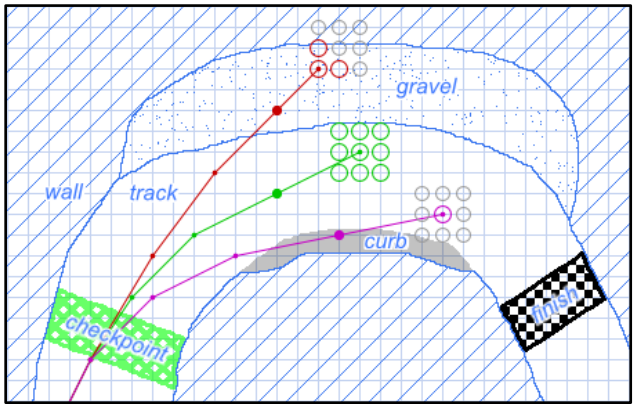


Figure 1 – three racers driving over three material types (gravel, track, and curb) in *Vector Racer*, labelled by name and showing the effect of each one.

Vector Racer visualises velocity selection as shown in **Figure 1**. This allows players to focus more on gameplay than mental calculations of where their race car will drive to, which is required in *Inertia Racetrack*. While manually calculating the race car's next position does provide a challenge, it is an artificial one and may frustrate players or make them bored[10]. Due to this, it is better to have a visualisation to hasten this process.

In *Vector Racer*, players can choose to reset their car from their last position after crashing or quit the race, while in *Inertia Racetrack* the game ends entirely. Ending the game may be fine when racing alone but can be frustrating when racing others. Allowing players to choose whether to leave without ending the race would make the experience smoother and less annoying for those who would like to continue[11].

In both games, all cars have the same acceleration and velocity limitations, noticeable in **Figure 1**, conforming to standard *Racetrack* rules. Having vehicles of varying performances would be intriguing and add a level of skill expression and adaptability to certain tracks for more refined gameplay[11].

Both games also allow cars to be in the same spot. This may be because the *Racetrack* game involves a discrete grid which makes these overlaps likely. It would be interesting to forbid cars being on the same intersection as an added challenge, as players would have to think about other players' positions and velocities in addition to their own[11].

(section continues)



Figure 2 – the list of race options available in *Vector Racer* when selecting a track.

Aside from gameplay, both games list the available tracks that are part of the game or made by other players. In addition to game tracks and player tracks, *PolyTrack* also lists the tracks the player themselves has made, useful for **Primary Objective 4.2**. Players can see details of tracks from these lists, such as the names and previews, to decide whether to play the track. In *Vector Racer*, players can also choose race options, such as the number of players to have and whether there are AI racers, as depicted in **Figure 2**.

2.2.2. Primary Objective 4.2

"Enable players to create tracks"

Both *Vector Racer* and *PolyTrack* include track building features.

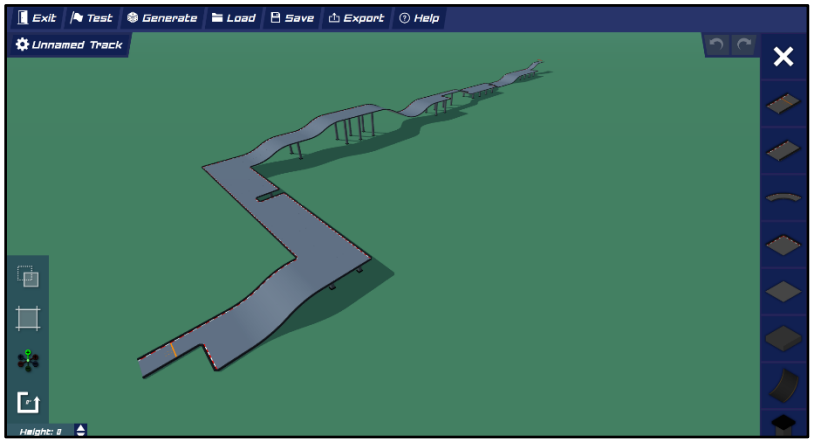


Figure 3 – the track builder from *PolyTrack*, including an auto-generated track and the available track parts (right) grouped by part category (right-most list).

PolyTrack has an in-game track builder with preset components, as shown in **Figure 3**. Meanwhile, *Vector Racer* requires players to upload a colour map which is converted to a track. *Vector Racer* provides more freedom since players can draw any kind of shape, while *PolyTrack* is more well-formed by having physical track parts and an unlimited building area. The former has its strengths in creativity, and the latter has its own in consistency and ease of placement[11].

PolyTrack also includes an option to auto-generate tracks, made possible with its track parts, which could make the building process easier[11]. It also includes options to undo or redo placements, edit a track's details, and save or export it. An in-game editor makes it much easier to assess what the track will look like in the game. The ability to test without uploading the track can save time and frustration to focus more on gameplay[11].

While *Inertia Racetrack* does not include any inherent building tools, its tracks do have decorations. This would be a fun element to include to make tracks look more appealing or for practicality[11], such as directional arrows so players know where to go.

2.2.3. Primary Objective 4.3

"Enable players to manage their tracks"

PolyTrack is the only game that allows players to manage their tracks. Players can view their tracks and edit their details, open the track itself for further editing, or delete them entirely.



Figure 4 – the custom tracks section in *PolyTrack* listing a single, player-made test track.

This is useful for players to track their creations to adjust or share with friends, and the option to remove them allows them to declutter their list. Players can also search tracks by name or author in each section. This system, as shown in **Figure 4**, would be helpful to include in the project for centralised track management.

2.2.4. Primary Objective 4.4

"Enable players to race against other players locally"

Local racing ensures players can enjoy the game with friends without an Internet connection. Both *Inertia Racetrack* and *Vector Racer* only allow a limited number of local players. Increasing the player limit for races offers more freedom to enjoy the game how they would like[11]. However, it is also important to consider that too many players may flood the screen and make the track hard to view.

(section continues)



Figure 5 – the replay system in *PolyTrack* playing back four players' races alongside each other.

Meanwhile, *PolyTrack* has replays and ghost racing, as demonstrated in **Figure 5**. It would be intriguing to implement replays, but the same level of skill, precision, and timing required in *PolyTrack*, being a game requiring arrow keys rather than discrete vectors for movement, would not apply. Due to this, replays would make competition trivial as players can watch them and copy the optimal path. Moreover, local and online multiplayer will likely cover the competitive spirit replays have, making them somewhat redundant. However, races against the player's personal best could be a fairer alternative that still allows players to analyse previous races and perform better[11].

2.2.5. Primary Objective 4.5

"Enable players to race against AI racers"

Vector Racer lets players race against AI, allowing players to enjoy the game with some challenge when there is nobody else to play with. However, it automatically fills all remaining player slots when selected. An option to adjust how many AI racers to include would provide more freedom to adjust gameplay to their liking[11], such as in cases where players may only want a few AI racers.

In general, it is also important to make considerations for AI where they would be for humans, as they may apply to them. For example, as mentioned in **Section 2.2.1**, AI racers could also be given a car type just as players may be able to choose their own. There could also be options to limit which cars AIs can have to tailor the experience or go against a certain type of race car, providing even more freedom and challenge[11].

2.3. Secondary Objectives Relations

This section assesses and compares each game based on each **Secondary Objective**.

2.3.1. Secondary Objective 1

"Enable players to race against players online"

Interestingly, none of the three games implement a direct form of online multiplayer, which is a new feature the project can incorporate.

(section continues)

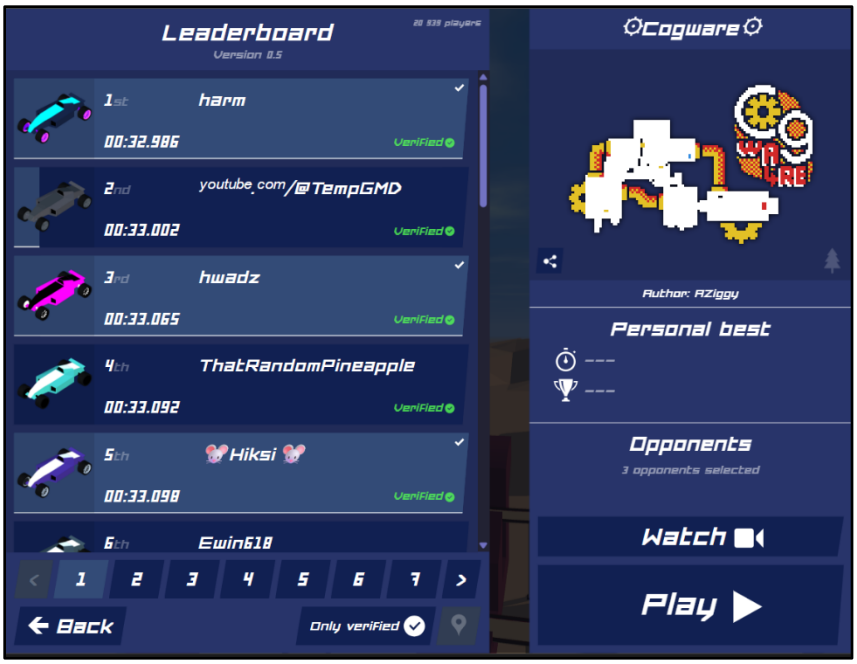


Figure 6 – the track details page in *PolyTrack*, showing all race attempts and their time
Three entries are selected which can then be watched or played against.

Vector Racer and *PolyTrack* both have paginated time leaderboards for each track, portrayed in **Figure 6**. This contributes to a competitive atmosphere by giving players benchmarks to work towards, making it a fun element to include in the project[11]. However, the games' leaderboards are lengthy but have no search capabilities. It would be good to include querying options for faster or specific searching rather than navigating each page.

2.3.2. Secondary Objective 2

"Enable players to race against AI racers of varying difficulty"

Vector Racer has AI with varying difficulties, which can be fun to play against by providing ranges of challenge[11].

1.	player 4	30.029		(#14)	(level 3 ai)
2.	player 1	30.171		(#18)	(level 3 ai)
3.	player 2	30.171		(#18)	(level 2 ai)
4.	player 3	30.171		(#18)	(level 1 ai)
5.	player 5	30.171		(#18)	(level 2 ai)
6.	player 6	38.029	(+3)	(#150)	(level 1 ai)

Track record: 28.533 all scores »

Figure 7 – the results of six AIs in *Vector Racer* after a session.
High-level AIs tend to perform better than low-level ones.

These difficulties are defined as levels, listed in **Figure 7**. However, players cannot specify their difficulty. This would be good to include for players to tailor a race that feels manageable for a fairer challenge[11]. It is unclear what exactly determines AI levels in *Vector Racer*, but it is important in any case to represent it in a way that players can understand its effect and the impact on gameplay so they can adjust the levels accordingly.

2.3.3. Secondary Objective 3

"Enable players to personalise their gameplay experience"

Settings allow players to adjust game elements to what suits them. *PolyTrack* has such options, but most will likely not apply to the project, such as key bind controls for vector selection, as only the mouse would need to be used. However, other settings, like sound and volume, will still be relevant. These settings must be considered in the game's design to ensure they are purposeful[4].

PolyTrack also has a personalisation section that lets players change the colours of their race car. This makes the game more fun and provides a sense of individualisation so players can race in their style[11]. However, the shape of the car itself is the same for all players and cannot be changed, which may be to ensure fair gameplay. Since the *Racetrack* game represents racers as points, this constraint does not necessarily apply, so it would be interesting and possibly engaging to include this customisation[11].

2.4. Other Games

As touched on in **Section 2.1**, other games were also researched. A notable one is *GPixel*[12], which has similar mechanics to the *Racetrack* game but uses speed rather than velocity and an eight-spoked steering wheel for direction. It represents the car's speed as a gear box, where each gear is a speed value. This could be a useful visual for players to assess how fast they are driving overall. *Gpixel* also includes weather and track conditions that alter visibility and car traction, which could be an interesting feature to incorporate for a wide range of track experiences and challenges[11].

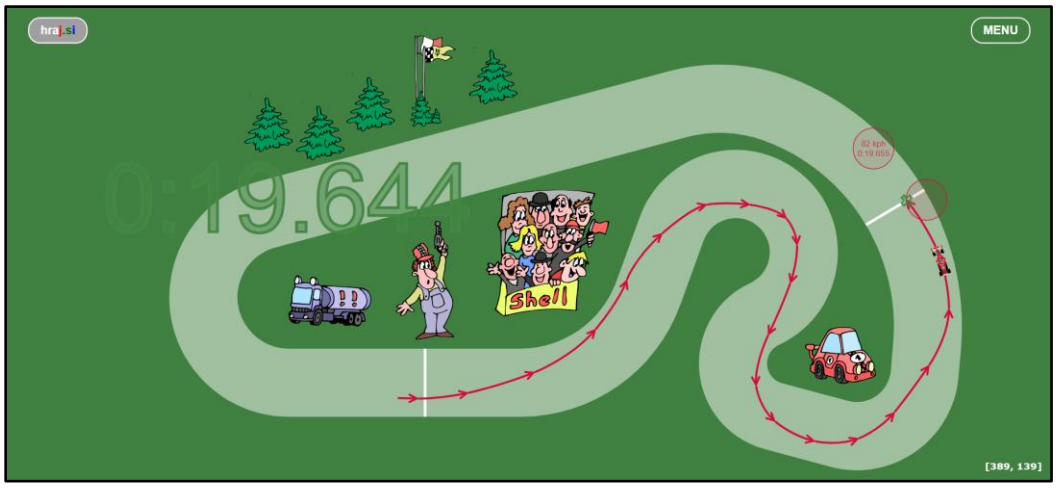


Figure 8 – a race from *Formulky*, demonstrating the Bezier curve movements around the track. The current time is also displayed when passing a white marker.

Another researched game was *Formulky*[13]. It is like the *Racetrack* game but deals with a non-discrete field and players drive according to Bezier curves, as shown in **Figure 8**. While the *Racetrack* concept uses a discrete grid, moving cars in the form of a curve would be interesting to include. It is important to consider how this might work with collisions.

3. Development Tools and Frameworks

This section explores the tools and frameworks which may be suitable for the project, followed by an analysis of which ones will be intended to use.

3.1. Development Areas

At a high level, the application can be split into three components, corresponding to the areas of full stack development[14]:

- **Front end:** handle player interactions, such as UI and racing controls
- **Back end**: handle server-side processes, like CRUD operations and multiplayer routing
- **Databases:** persistently store racing or player data, primarily built tracks, race times, and customisations

3.2. Technology Decisions

3.2.1. Front End and Rendering

Standard HTML, JavaScript, and CSS will be used for the front end. While Phaser[15], Kaboom[16], and p5.js[17] are libraries suitable for web-games and rendering, they utilise a loop which updates the game per frame. Since the Racetrack game is a turn-based game more dependent on player-driven events than continuous changes, it seems redundant to use them. Frameworks like React.js may be initially complex but could be considered for later versions, especially for its benefits in reusability of code and UI elements[18], which could be useful for track lists or leaderboards. As the concept itself is simple, HTML and JavaScript together seem sufficient to achieve gameplay based on the objectives, especially with JavaScript's extensive range of libraries.

3.2.2. Back End

Given that JavaScript will be used to program the front end, it seems sensible to use Node.js[19] for the back end. Though languages like PHP are dedicated server-side languages[20], using JavaScript across the application may make the development process easier and save having to learn another language in the interest of time. While Node.js is less mature than PHP[19,20], it seems equally capable in achieving the objectives making use of JavaScript libraries, making it dissimilar from PHP in fulfilling the project's needs.

3.2.3. Databases and Firebase

Firebase[21] will be used for databases. In addition to its NoSQL[22] databases, this is also because of its versatility in the tools it provides, primarily its account authentication and hosting tools, which are useful to assign data to players and implement multiplayer with a client-server model. MySQL[23] would be suitable for storing sensitive information, such as login details. However, as the project grows, there may be several instances where fields are added or removed. NoSQL's "schema-less" document-based nature can better accommodate these changes, making it more suitable for forwards and backwards compatibility[22]. Additionally, some data, like tracks and customisations, may be more appropriate and easier to capture with documents. While MongoDB is also a NoSQL database[24], it is a standalone tool which does not include the additional benefits Firebase has. Firebase's server hosting tools can also provide security by hosting Node.js servers elsewhere to keep them from clients[21].

3.2.4. Multiplayer Networking

As for multiplayer capabilities, hosting on Firebase would form a client-server network architecture, where the server can be used as a bridge between clients[25]. Socket.IO is a common JavaScript library which uses the WebSocket protocol to enable bidirectional event-based communication between a client and server[26]. This makes it a suitable for multiplayer as it would allow players to provide inputs or requests to the server, which can then replicate the change to other clients or provide the relevant data. Games also commonly use matchmaking features such as lobbies so players can create games[27], which Socket.IO can similarly be used for.

3.2.5. Game Engines

Game development platforms like Unity[28] are powerful and well-suited for complex games and include several libraries to cover all bases[29]. However, most of its features will likely not be used to their full potential for the project, most notably its rendering and modelling capabilities[28], due to the simplicity of the *Racetrack* concept[1].

Moreover, there may be a lot of unnecessary overhead when setting up. From previous experience, it has been found to be quite cumbersome compared to web technologies, particularly around its script and component structure, which took a lot of time to learn. Given these points, it seems better to forego it for the project in favour of web technologies, with which there is more experience, despite its use cases.

3.2.6. Further Considerations

These technology choices are based on research and previous experience in different contexts, and there are many more frameworks and tools than what has been mentioned here. Further into the project, it may be found that some of these are not as suitable as first thought, are entirely incompatible, or have undesirable limitations. Equally, other technologies not initially researched may be used instead. For instance, Firebase has a free and paid plan which has not been fully explored yet: depending on the outcome, MongoDB may need to be used instead, which has a limited database size but not limited requests. To prepare for this possibility, decoupling code methods from the technologies would be helpful to allow new tools can be swapped without having to modify its dependents[30].

4. Application Requirements

Based on the researched game and available tools, a list of requirements can be collated to capture the desired features and improvements to be included in the project' implementation.

4.1. Requirements Glossary

The following glossary explains terms used in the requirements.

Term	Definition
Game	The application the project aims to produce.
Track	A track which players can race around.
Track Details	The details of a track: name; description; creation date; author; dimensions; and tags.
Race	An instance of a session where a set of players race around a track.
Race Car	The vehicle a player uses to race around a track, which may or may not be a "car".
Starting Line	The line on the track where all players start (and end) the race from.
Checkpoints	Points on the track that mark where players should go.
Materials	Traversable track elements that affect racer movements for the current turn if on it.
Obstacles	Track elements that cause players to crash when driven into.
Decorations	Ornamental track elements which have no effect and are solely visual.
App Settings	Player settings for the game: volume; and music.
Velocity	A racer's current vector offset per turn.
AI Racer	A non-human racer.
Race Options	A set of settings to apply to a race: player count; AI included; AI count; player race car types (for local sessions); and AI race car types.
Race Details	A set of live gameplay information: player velocities; turn count; player paths; and player checkpoint passes.
Local Multiplayer	Several players racing together on the same device.

Online Multiplayer	Several players racing together on separate devices.
Ghost	A representation of a player's personal best time and its path.
Lobby	A room players can join online to begin a race together.
Leaderboard	A list of player track completion times for a track.

4.2. Requirements List

The table below captures these requirements as user stories[43] in the Azure DevOps[31] backlog for the project[45]. Orange, purple, and white rows represent "Epics", "Features", and "User Stories" respectively[44]. Each row includes the following details:

- **ID:** the unique ID of the user story in the Dev Ops project with a corresponding link.
- **Objective:** the objective the user story relates to (where "P" is primary, and "S" is secondary)
- **Title:** the concept the user story covers, including MoSCoW phrasing[32].
- **Points:** the anticipated size of the story on a scale relative to other stories.

Note that the MoSCoW wording[32] decides whether including a requirement is needed for the project's success, not its corresponding objective.

V0 and V1 are planned to be completed before the interview week to demonstrate a version including all primary objectives (see **Section 7.2**). Subsequent versions' requirements are indicative, as these may change depending on the state of the project closer to the time. Any features devised after the Interim Report is submitted will be analysed and categorised into a fourth version (V4) if they are large or incorporated into existing epics if they are small enough.

ID		Title	Points
351		Version 0 (MVP for Core Gameplay)	**259**
458		[V0] - Core	**8**
458	P4.1	[V0] - The player MUST be able to start the "Game"	8
361		[V0] - "Track" Viewing & Selection	**39**
431	P4.1	[V0] - The system MUST show all available "Tracks"	13
432	P4.1	[V0] - The player MUST be able to choose a "Track" to play	13
436	P4.1	[V0] - The system SHOULD show "Track Details"	13
365		[V0] - "Track" Representation	**55**
434	P4.1	[V0] - The system MUST be able to represent "Tracks"	55
377		[V0] - "Race" Gameplay	**149**
483	P4.1	[V0] - The system MUST show the "Track" the player chose to play	34
476	P4.1	[V0] - The system MUST show the player on the "Track"	13
433	P4.1	[V0] - The player MUST start a race from the "Starting Line"	13
447	P4.1	[V0] - The player MUST be able to control their "Race Car" when racing	21
487	P4.1	[V0] - The player MUST be bound to the "Track" confines when racing	21
448	P4.1	[V0] - The player MUST be able to complete an ongoing "Race"	34
497	P4.1	[V0] - The system SHOULD show the "Race Results" when completing a "Race"	13

ID		Title	Points
372		[V0] - "App Settings"	**8**
435	S3	[V0] - The system MUST show all current "App Settings"	8
341		Version 1 (Building, Extended Gameplay, and Basic Saving)	**840**
364		[V1] - "Track" Building - Controls	**110**
439	P4.2	[V1] - The player MUST be able to save the "Track" they're building	21
586	P4.2	[V1] - The player SHOULD be able to undo building actions	21
587	P4.2	[V1] - The player SHOULD be able to redo building actions	21
446	P4.2	[V1] - The player SHOULD be able to set the "Track Details" of the "Track" they're building	13
588	P4.2	[V1] - The player COULD be able to test their "Track" while building it	34
522		[V1] - "Track" Building - Path	**76**
513	P4.2	[V1] - The player MUST be able to build their own "Track"	34
444	P4.2	[V1] - The player MUST be able to adjust the path of the "Track" they're building	21
538	P4.2	[V1] - The player COULD be able to define "Checkpoints" for the "Track"	21
523		[V1] - "Track" Building - "Materials"	**89**
521	P4.2	[V1] - The system SHOULD show all available "Materials" when building a "Track"	21
511	P4.2	[V1] - The player SHOULD be able to place "Materials" on the "Track" they're building	34
454	P4.2	[V1] - The player SHOULD be able to adjust placed "Materials" on a "Track" they're building	21
515	P4.2	[V1] - The player SHOULD be able to remove placed "Materials" from a track they're building	13
525		[V1] - "Track" Building - "Obstacles"	**89**
526	P4.2	[V1] - The system SHOULD show all available "Obstacles" when building a "Track"	21
527	P4.2	[V1] - The player SHOULD be able to place "Obstacles" on the "Track" they're building	34
528	P4.2	[V1] - The player SHOULD be able to adjust placed "Obstacles" on the "Track" they're building	21
529	P4.2	[V1] - The player SHOULD be able to remove placed "Obstacles" from the "Track" they're building	13
524		[V1] - "Track" Building - "Decorations"	**89**
520	P4.2	[V1] - The system COULD show all available "Decorations" when building a "Track"	21
437	P4.2	[V1] - The player COULD be able to place "Decorations" on the "Track" they're building	34
452	P4.2	[V1] - The player COULD be able to adjust placed "Decorations" on the "Track" they're building	21
451	P4.2	[V1] - The player COULD be able to remove placed "Decoration" from the "Track" they're building	13
411		[V1] - "Track" Management	**89**
559	P4.3	[V1] - The system SHOULD show the "Track Details" of the "Track" they've built	13
440	P4.3	[V1] - The player SHOULD be able to edit the "Track Details" of "Tracks" they've built	21

ID		Title	Points
441	P4.3	[V1] - The player SHOULD be able to delete "Tracks" they've built	13
419	P4.3	[V1] - The player SHOULD be able to save copies of "Tracks" they've built	21
422	P4.3	[V1] - The player COULD be able to set "Tags" for "Tracks" they've built	21
344		[V1] - "Track" Viewing & Selection	**81**
445	P4.1	[V1] - The system MUST show all a player's saved built "Tracks"	21
450	P4.1	[V1] - The player COULD be able to filter all available "Tracks" by keywords	13
512	P4.1	[V1] - The player COULD be able to filter all available "Tracks" by their own	13
405	P4.1	[V1] - The system COULD show a preview of a "Track"	34
535		[V1] - Extended "Race" Gameplay	**89**
534	P4.1	[V1] - The system SHOULD affect players driving over "Materials"	34
536	P4.1	[V1] - The system SHOULD crash players driving into "Obstacles"	34
565	P4.1	[V1] - The player SHOULD be able to choose whether to continue a "Race" after crashing	21
386		[V1] - "Race Car" Selection	**60**
516	P4.1	[V1] - The system SHOULD show all available "Race Cars"	13
517	P4.1	[V1] - The player SHOULD be able to choose a "Race Car" to race with	13
518	P4.1	[V1] - The player SHOULD start a "Race" with their selected "Race Car"	13
572	P4.1	[V1] - The system SHOULD constrain player movement according to "Race Car" type	21
425		[V1] - Live "Race Details"	**68**
537	P4.1	[V1] - The system COULD show player paths throughout a "Race"	21
453	P4.1	[V1] - The system COULD show the turn count during "Races"	13
426	P4.1	[V1] - The system COULD show the player's current "Velocity"	13
427	P4.1	[V1] - The system COULD show "Checkpoints" the player has passed	21
332		Version 2 ("AI Racers", and "Local Multiplayer")	**267**
375		[V2] - "Race Options"	**34**
560	P4.1	[V2] - The system SHOULD show "Race Options" when selecting a "Track"	13
561	P4.1	[V2] - The player SHOULD be able to set "Race Options"	21
346		[V2] - "AI Racers"	**136**
568	P4.4	[V1] - The system SHOULD show "AI Racers" on the "Track"	13
562	P4.4	[V2] - The system SHOULD make "AI Racers" take optimal moves in "Races"	55
563	P4.4	[V2] - The system SHOULD apply "Material" effects to "AI Racers"	34
564	P4.4	[V2] - The system SHOULD apply "Obstacle" crashes when "AI Racers" drive into them	34
566		[V2] - "Local Multiplayer"	**63**
569	P4.5	[V2] - The player SHOULD be able to race with any number of "Local Players"	21
571	P4.5	[V2] - The player SHOULD be able to set the "Race Car" types of "Local Players"	21
567	P4.5	[V2] - The system SHOULD show the player and "Local Players" as distinct "Race Cars" on the "Track"	21

ID		Title	Points
589		[V2] - "Ghost" Racing	**34**
590	P4.5	[V2] - The player COULD be able to race against their personal best for a "Track"	34
366		Version 3 ("Online Multiplayer", "Race Car Customisation", and Extended Data Management)	**377**
573		[V3] - "Track" Sharing	**89**
574	P4.1	[V3] - The system COULD show "Tracks" built by other players	55
575	P4.1	[V3] - The player COULD be able to play other players' "Tracks"	34
591		[V3] - "Track Leaderboards"	**68**
592	S1	[V3] - The system COULD save the best player times for a "Track"	34
593	S1	[V3] - The system COULD show all player times for a "Track"	34
396		[V3] - "Lobby" Management	**123**
576	S1	[V3] - The player COULD be able to create "Lobbies"	13
577	S1	[V3] - The system COULD show all available "Lobbies"	21
578	S1	[V3] - The player COULD be able to join an available "Lobby"	21
580	S1	[V3] - The system COULD show players present in a "Lobby"	21
579	S1	[V3] - The player COULD be able to leave a "Lobby"	13
581	S1	[V3] - The player COULD start a "Race" from a "Lobby"	34
582		[V3] - "Online Multiplayer"	**55**
585	S1	[V3] - The player COULD be able to race against other players online	55
397		[V3] - "Race Car Customisation"	**42**
583	S3	[V3] - The system COULD show the player's current "Race Car" appearance	21
584	S3	[V3] - The player COULD be able to edit their "Race Car" appearance	21

The project requirements are worth **1,743** story points in total as they are at the time of writing. The densest version is V1 due to most features concerning extended track building and listing capabilities, which will both require moderate code infrastructure to prepare. However, some features are similar and overall may need less work. For example, as discussed in **Section 5.3**, obstacles and decorations are based on the same concept as materials, so the same code could be duplicated and slightly modified.

5. Implementation & Design

Based on the requirements, the application can largely be grouped into the following areas: “Track Building”; “Track Listing, Selection, and Management”; “Game Creation and Race Options”, “Race Gameplay”, “Car Customisation”; and “App Settings”. Implementations and designs have been provided for each area in the upcoming sections.

5.1. Client-Server Architecture

As discussed in **Section 4.2**, Firebase will be used to host the web application on a server. This will allow Node.js scripts to be separated from clients.

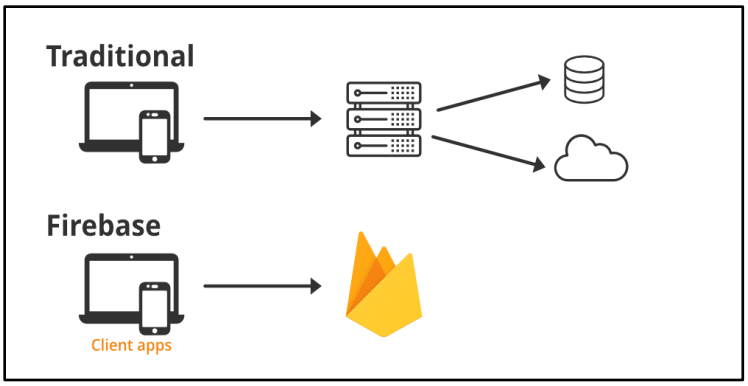


Figure 9 – a diagram of the role Firebase plays in back-end management, bundling the servers and databases[33].

Like typical server models, the server will act as a bridge between the client and database, as well as other clients. **Figure 9** shows the basic architecture of Firebase[33]. The server will be responsible for storing available games and game states. Additionally, all validation will be handled by the server. This includes clients sending their gameplay inputs, like vectors, to the server: if an input or submission is invalid, the server will respond to the client accordingly.

5.2. Screen Navigation

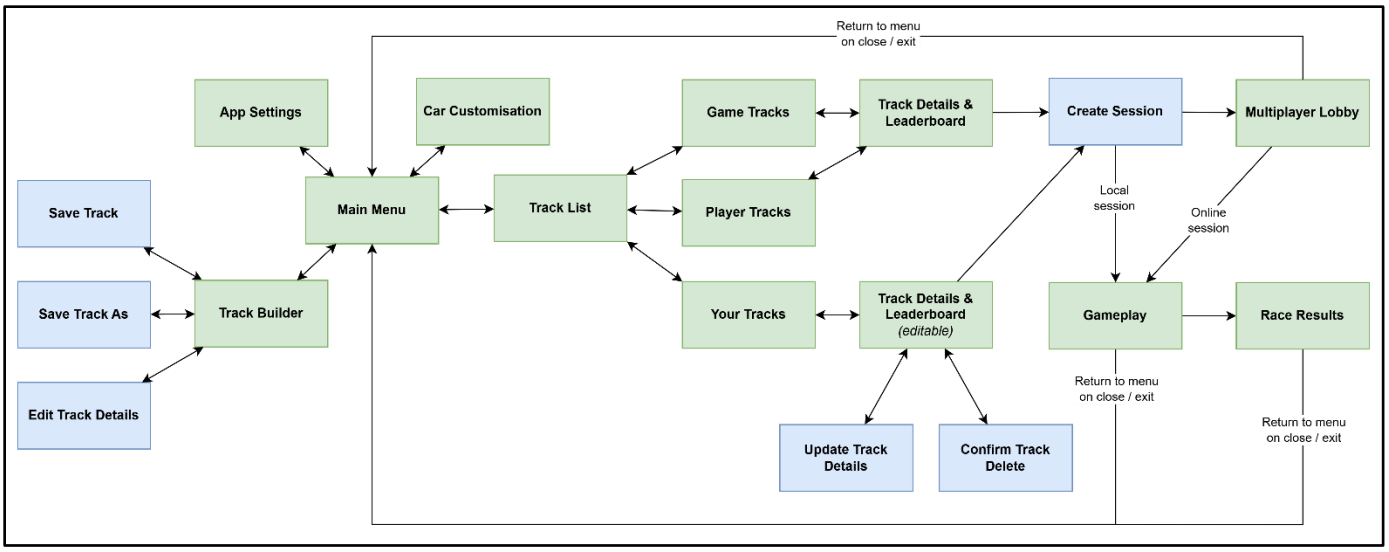


Figure 10 – a flowchart showing all screens (green) and popups (blue), and how to navigate to them. Bidirectional arrows represent returning as well.

Figure 10 holistically shows how these sections can represent linked screens. The main menu is the application root, commonly used in many games like those researched[7,8,12]. A main menu allows players to navigate between all areas of the application through a centralised point and can help familiarise themselves with the game or ground them if they become lost[34]. These screens are detailed in the following sections.

5.3. Track Building

Building is the most fundamental aspect of the game, as the way tracks are represented must be simple enough for players to reproduce using in-game tools. This form also determines how the application handles gameplay interactions and calculations between players and tracks. Hence, it has been discussed first.

5.3.1. Racetrack Representation

From the researched games, there are two ways of creating tracks: one being with an image map[8]; and the other with interconnected parts[9]. Methods beyond these games mostly involve arranging track paths as a series of nodes. Altogether, these methods can be summarised as sequences of nodes and 2D arrays, considering that images are essentially arrays of pixel colours.

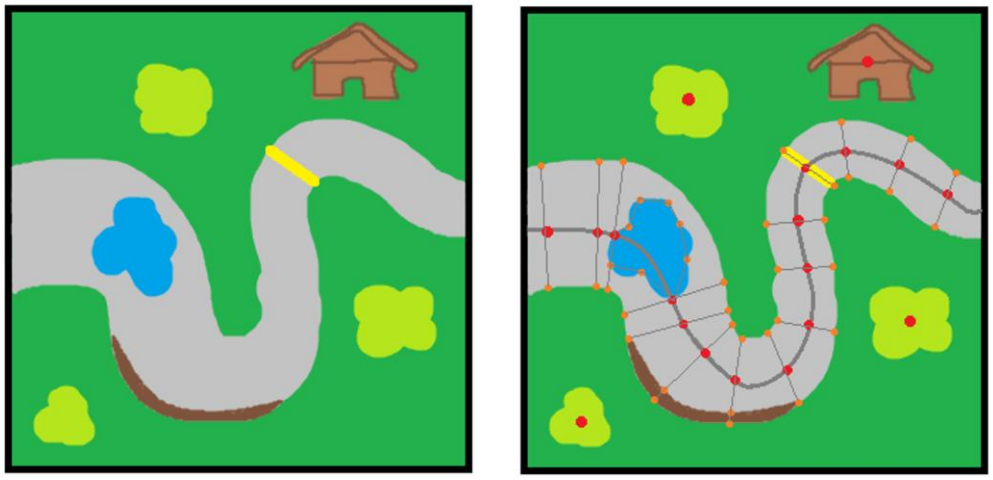


Figure 11 – a section of a drawn track (left) with its spline-based counterpart (right). Note the spline's simplicity and difficulty in representing the track shape and materials respectively, and conversely for the drawn track.

Figure 11 depicts the same track represented as a drawing and as a spline. Using a series of nodes is much more memory efficient than representing a track as a 2D array since data is only required for the node positions, as opposed to a colour corresponding to each pixel. Moreover, the path between the nodes can be rendered as a spline[35] for a more organic track. Splines can also capture details about the track, such as its total length and race car positions along it[35]. This could be used determine track direction and checkpoints more easily than a drawing, offering simpler validation against wrong-way driving. While free drawing does provide more freedom for defining the specific width of the track, a width can be assigned to each node to determine the thickness of the track at specific points. Finally, the series of nodes can either be open-ended or form a closed loop for simple straight tracks or complete circuits, and more nodes can be included for higher precision in bends.

However, adding materials to the track is a factor to consider. For a 2D array, each cell could be painted and applied a track or material indicator and rendered accordingly: with nodes, this is more complicated. Materials could be seen as a component of the track itself, as demonstrated by the dirt patch on the edge of the track in **Figure 11**. In this way, they could be represented as layered nodes. However, this does not work particularly well for materials in the middle of the track, as with the puddle in the same image. Moreover, if the track form is changed, the material might also move inconsistently. It would be simpler to isolate materials from the track altogether and have them as separate polygons, as this would be simpler to create and independent of the track if it were adjusted.

Another element is detecting race cars on top of materials, essential to affect player movement in the next turn. A 2D array can simply be indexed with the player's position to determine whether they are on a material or obstacle. However, with nodes, calculations are required to determine track and material extents, then compared to the race car's position. This is detailed in **Section 5.6.1**. Likewise, what applies to materials also applies to obstacles, except the latter determines crashes. Decorations are purely ornamental and have no effect: they could be treated as single nodes with a rendered image on top, or polygons of their own.

Despite the relatively complex workaround to these limitations, it is still worth using the node approach over the 2D array for the sake of efficient storage. However, care must be taken to ensure a sufficient UX with supportive

tools to place nodes and create polygons easily. Additionally, selected points on the grid must be mapped appropriately to materials on the track to correctly affect velocity or cause crashes. Tracks represented by series of nodes also invites the possibility of auto-generating tracks, which may further alleviate the required effort to build tracks[11].

5.3.2. Track Builder Interface

A section in the form of a track builder will allow players to create tracks. In line with the structure mentioned in **Section 5.2.1**, players will use nodes and polygons to construct their tracks.

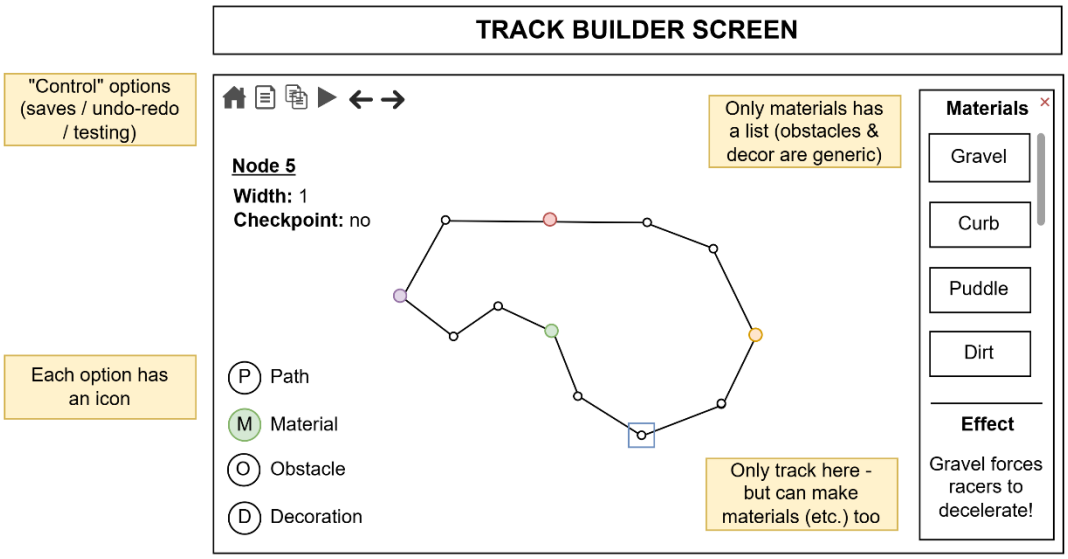


Figure 12 – a design for the track builder. The material list is showing, and the 6th node of the track is selected. The starting line (purple) and checkpoint (orange) are showing.

Figure 12 shows a design for the interface. The image shows a straight path for simplicity, but real tracks will use splines and form smoother bends. Since obstacles and decorations have no type like materials do, a single option to draw an obstacle or decoration from a generic polygon is sufficient.

5.4. Track Listing, Selection, and Management

Players will be able to view all available tracks that are part of the game, made by other players, or made by themselves, like the design used by *PolyTrack*. Each section allows players to view the track details and start sessions from them. It will include each track list, details for each track, track leaderboards, and management options for the player's own tracks.

5.5. Game Creation and Race Options

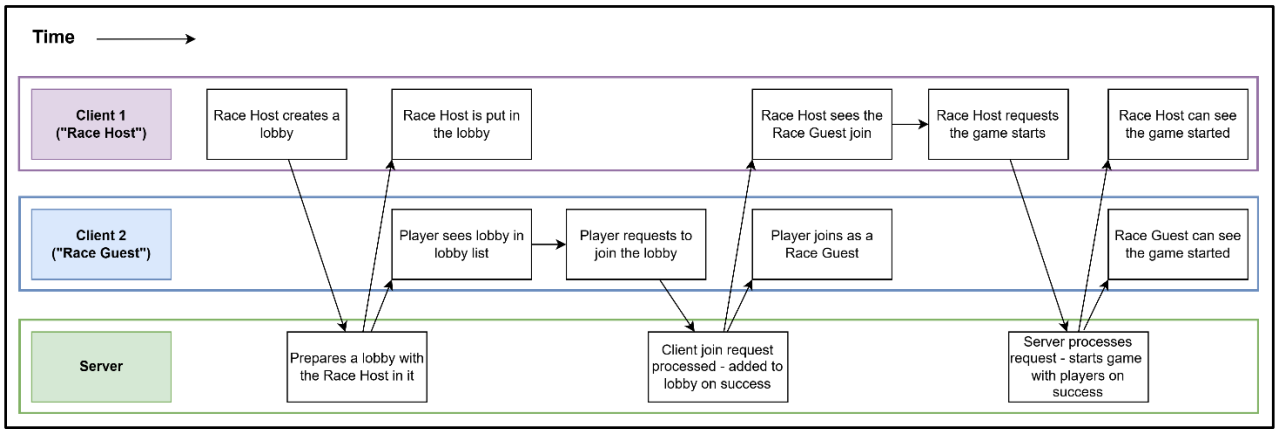


Figure 13 – a flowchart depicting lobby creation and joining for online multiplayer sessions.

Players will be able to create a local session, or a lobby for online multiplayer games which other players can join. Local sessions require no server operations to set up and can simply run on the client. However, online multiplayer sessions need the server to manage lobby creation, player joining, and propagation to other clients, as shown in **Figure 13**. As with joining, leaving will also be handled accordingly.

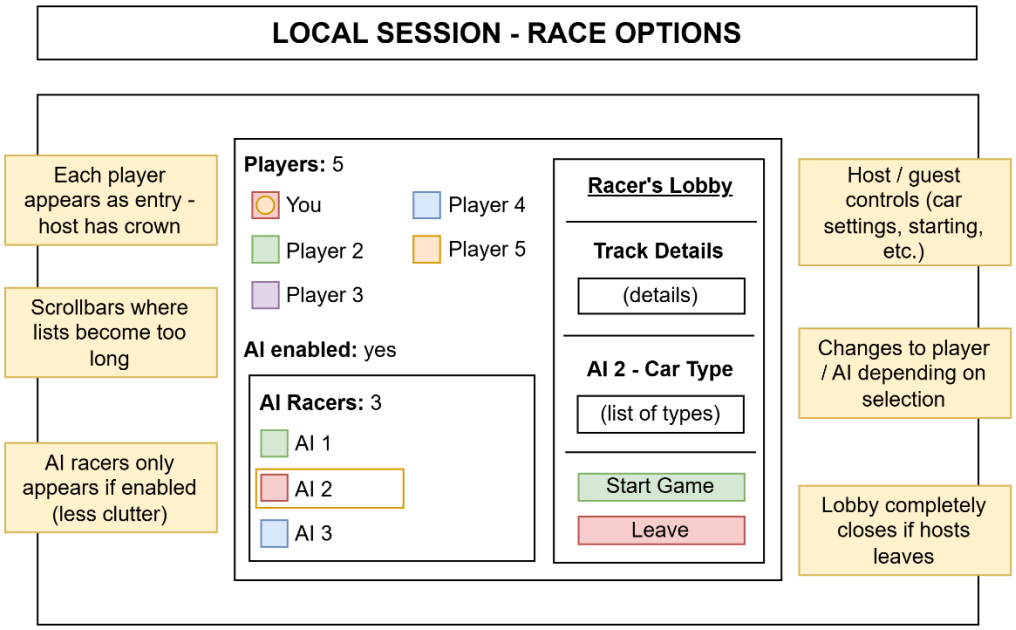


Figure 14 – a design for local session options, showing all players and lobby details.

Players will also be able to set race options. **Figure 14** depicts the options for a local session in the form of a lobby. Multiplayer session options will be the same but not allow setting the car types of other players. If a player leaves from a multiplayer lobby, they are simply removed from it, whereas if the host leaves, the lobby closes and all players are returned to the main menu.

5.6. Race Gameplay

This section covers elements concerning core gameplay and its designs.

5.6.1. Move Selection

As in *Vector Racer*, players will race around tracks by selecting vectors visualised over a selection grid. The movements they can make are determined by their race car type and the material they are on top of, like **Figure 1** in **Section 2.2.1**. If players crash, they can choose to restart from their last location at zero velocity or end the race for themselves. Other players will still be able to continue until the race is over.

As mentioned in **Section 5.2.1**, players on materials or in obstacles must be detected. The ray casting algorithm[36] is one procedure to determine whether a point is inside a polygon. The method fits this context, as race cars are single points, and materials and obstacles are represented as polygons. Considering this, a full process to determine intersections with materials or obstacles to decide outcomes could be as follows:

```
// intel – gathered only once before the race starts
1. Map the track to the selection grid
2. Gather all grid points which can be driven to and over (i.e., track points)
3. Convert the gathered points to a graph

// application – performed each turn
5. For each obstacle on the field...
   a. Apply the ray casting algorithm to determine whether the player is in it
   b. If so, crash the player and stop here
   c. Otherwise, continue to Step 5

6. For each material, check if the player is in it
   a. Apply the ray casting algorithm to determine whether the player is in it
   b. If so, limit the player's movement options according to the material's effect
   c. Otherwise, allow the player to move as normal without limits
```

The algorithm has O(n) complexity, having to check the player's position for intersections against each polygon edge of all materials and obstacles. While it is sufficient for tracks with few or low-resolution polygons, there are ways to optimise it, such as using quadtrees. This may be considered further into the project.

5.6.2. AI Pathfinding

AI racers must determine an optimal track path to complete races. In the same way players choose a vector for their current turn, the AI will also select a velocity under the same constraints. Considering the grid is discrete and an overlay of the track, all viable driving points can collectively be viewed as a graph, where each node is a point on the track that can be reached and driven over without crashing. The AI will use the cell intersections to determine the material under the track to inform its decisions.

The optimal racing line is the path around the track a racer takes to complete it in the fastest time possible. Naturally, this corresponds to the shortest path around the track. Accounting for the graph representation of the track, there are many pathfinding algorithms, a prime yet simple example being Dijkstra's shortest path algorithm[37]. Furthermore, there are optimisations to this procedure, namely, to use a Fibonacci heap[38] as a priority queue.

On top of optimal pathfinding, it be good to include variation in the AIs performances to avoid making the AI too performant. This could be in the form of limiting path depth, where high-level AIs can plan much further ahead. However, this method alone would make races predictable, as the calculations would be the same each time. Other approaches include making low-level AIs take moves from the second or third most optimal path instead or simply including random variation. A mix of these methods would be best to ensure variation between races. Other methods include fuzzy logic[9], which may be investigated later in the project.

Based on this, an algorithm for an AI racer could be as follows:

```
// intel – gathered only once before the race starts
1. Map the track to the selection grid
2. Gather all grid points which can be driven to and over (i.e., track points)
3. Convert the gathered points to a graph

// action – performed each turn
6. Depending on the AI's level, limit its calculation depth
7. For each depth, constrain the AI's movement with the procedure from Section 5.5.1
5. Determine the "best" next move to make from its view
6. At random, based on the AI's level, deviate its final velocity by some offset within its acceleration range
```

5.7. Car Customisation

Players will be able to customise their car with colours and styles. However, it is important to ensure that players can discern the type of vehicle despite styles.

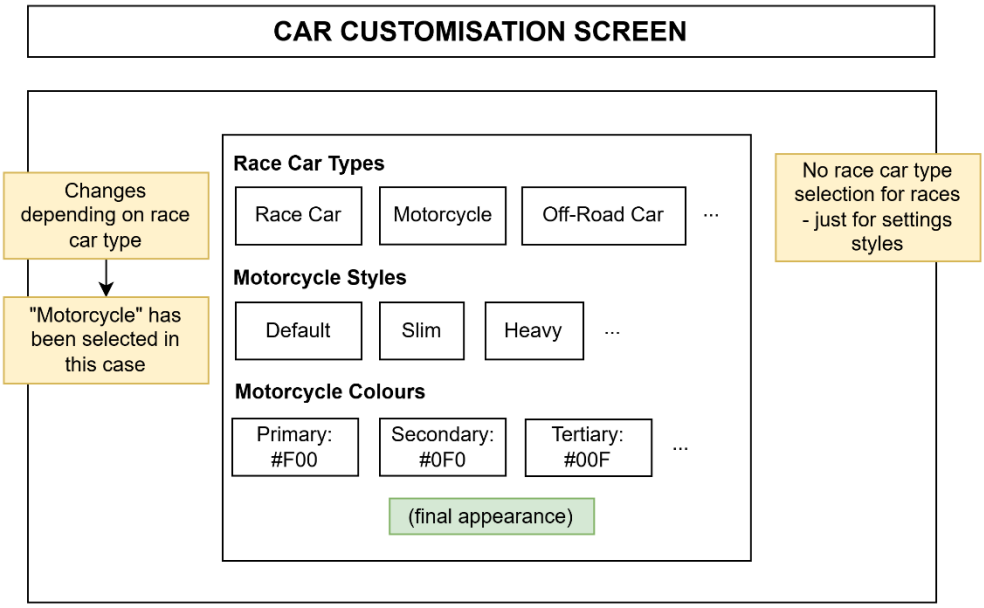


Figure 15 – a design for the customisation screen, listing all possible car types, styles, and colour options. The selection result is displayed at the bottom.

Figure 15 shows a view of the car customisation section. A similar style of player entries in **Figure 14** in **Section 5.4** could also be used as a list during the game to show player details so car types can be discerned.

5.8. App Settings

Finally, players will have access to application settings which they can adjust. This will be a simple screen with a presented as a simple list containing all options with appropriate sliders, checkboxes, and number inputs.

5.9. Game Handling and Data

5.9.1. Documents and Objects

Since Firebase uses NoSQL databases and stores data as JSON documents, it seems sensible to follow an object-oriented paradigm since objects and JSON are closely linked in that they are essentially a collection of key-value pairs.

Objects can be converted to JSON to be stored as documents. The document data can then be read and converted back to the original object form as before based on the keys and values. Given this, the following class diagrams are indicative of how the objects will be stored in the database.

5.9.2. Holistic Class Diagram

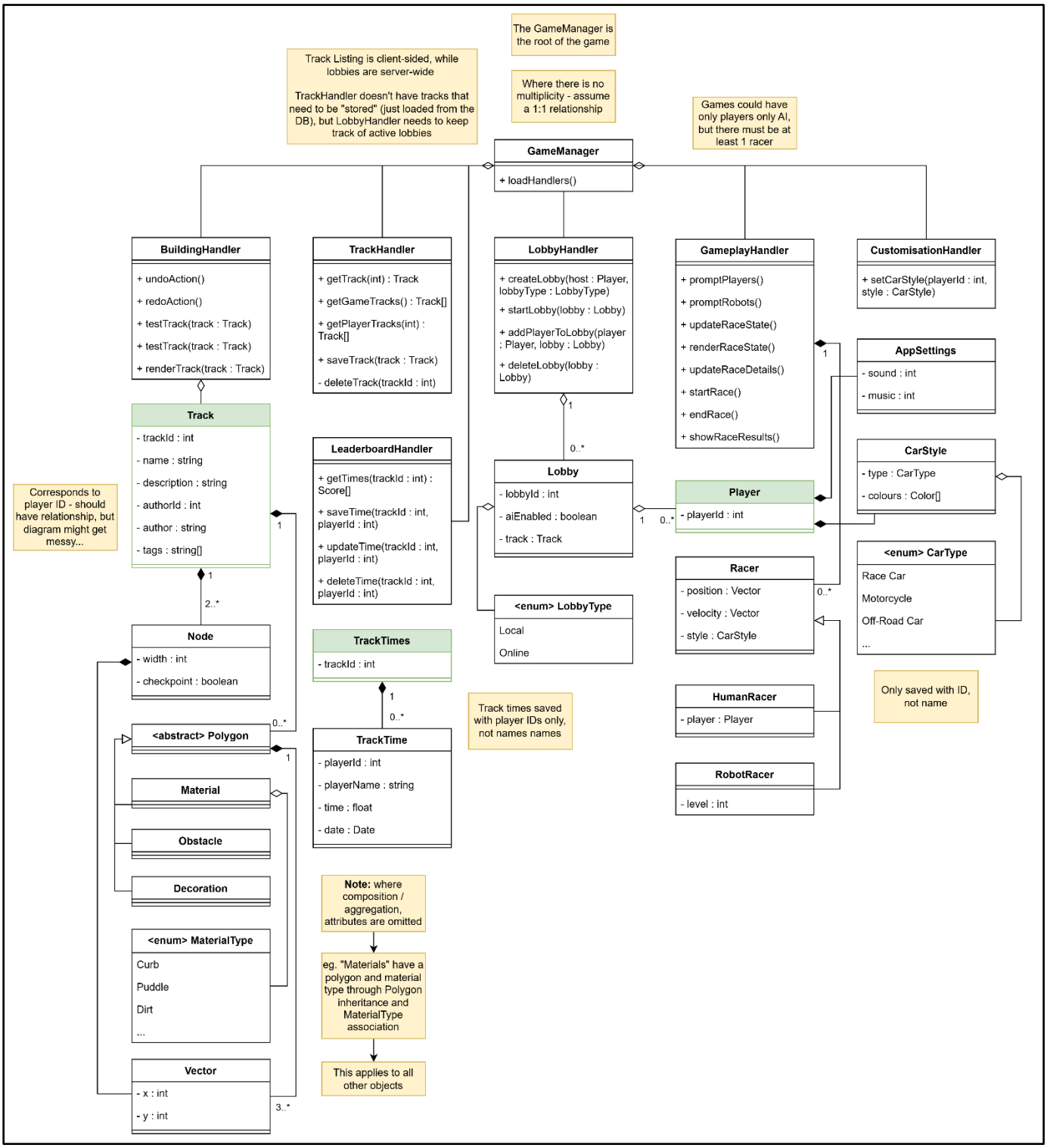


Figure 16 – a class diagram representing the most significant game components and their relationships.

Figure 16 shows a class diagram depicting the most essential elements of the game and the relationships between them, indicating what the game will have. The game has several data objects which hold information about an aspect of the game, such as `Lobby` for lobby data and race options, `Player` for player data, and `Track` for the full form and details of a racetrack. Meanwhile, handler classes perform all operations for the game and manipulate these data objects.

In line with the Law of Demeter[39], this division between data and functionality provides a cleaner approach as procedures related to an area are only managed by one common class. For example, to add a player to a lobby, the `addPlayerToLobby()` method of the `LobbyHandler` is invoked, rather than a `joinLobby()` method of the Player directly. The latter would likely require retrieving the lobby through the handler, then checking whether it is full, all from the perspective of a player, which may cause inconsistencies if there are other `joinLobby()` methods in other classes. The concept of handlers and data objects to manage them has been accomplished before[40] and seems like a sensible approach for game development to achieve this segregation.

Finally, all green classes in the diagram represent objects which will be saved. For example, `Track` objects can be saved with their path, polygons, and other data.

6. Application Testing

Since the main programming language for the project is JavaScript, it would be useful to use a corresponding library to test the code and ensure it functions correctly. One library which covers both unit and integrated tests is Vitest[41], a lightweight testing framework, which will be used for the project.

Unit tests will help ensure that previous functionality performs accordingly on an independent level, unaffected by any other code, while integration testing will ensure that functionality added on top of existing systems will work with it appropriately. Each user story will have a unit test, and each feature will have an integration test. Features across different epics may also have an integration test written for them for more assuredness.

7. Project Timeline

7.1. Current Progress

The table below outlines the tasks achieved and progress made before the submission of the interim report. Weeks in this table are relative to when work was first undertaken for the project. For more detailed entries, please see the Project Log in the project's GitLab repository[42].

Week #	Start Date	End Date	Tasks
1 – 12	16/07/25	29/09/25	• Preliminary project research (researching Racetrack concept, existing "VectorRacer" game, racetrack representations, frameworks and tools). • Prepared an Azure DevOps backlog with requirements.
12	30/09/25	30/09/25	• First supervisor meeting to discuss the selected project and share research done before start of term.
12	02/10/25	02/10/25	• More Gitlab file preparations. • Refining and categorising requirements in backlog.
12	03/10/25	03/10/25	• Documented expected requirements for the MVP version to continue and refine for the next supervisor meeting. • started drawing screen designs and requirements documentation for subsequent versions.
13	06/10/25	06/10/25	• Wrote some code to take snapshots of the Azure DevOps backlog in case it cannot be viewed.
13 – 14	07/10/25	17/10/25	• Researched different technologies to use for the app (front-end, back-end, databases, rendering / drawing).

			• Discussed project plan in supervisor meeting and requirements for feedback.
14 – 15	17/10/25	21/10/25	• Revised requirements and discussed in supervisor meeting and received more feedback.
15 – 16	22/10/25	31/10/25	• Discussed Interim Report (research and structure, timeline, approach, questions about structure, identifying aims and objectives, comparison with last year's project's report). • Discussed Project Plan feedback (title change, specify project timeline date).
16 – 18	01/11/25	11/11/25	• Interim Report updates (reorganising sections relative to objectives for improved flow and ease of linking)
18 – 19	12/11/25	20/11/25	• Reanalysing researched games and exploring new ones to cover missed areas • Researching server architectures and additional algorithms, like the ray casting algorithm and Firebase's architecture • Finalising Interim Report (designs, architecture, timelines, proofreading, etc.)

7.2. Workload and Future Timeline

As touched on in **Section 4**, the planned requirements amount to **1,743** story points. 8 points is the lowest of all requirements, which is expected to be half a day's work based on personal ability. This means around 109 days are required to fulfil all user stories. From the week after the Interim Report submission to the software due date (i.e., 24/11/25 – 27/04/26), there are 106 working days.

Based on the projected and actual dates, it seems possible to fulfil all requirements for a well-formed project. However, this is only an estimate, and time must also be spent doing final tests and writing the dissertation. Moreover, as mentioned in **Section 4**, some features may become much simpler once most code infrastructure is in place, which could put development ahead of the timeline at some points.

With these considerations in mind, the table below outlines a plan of the tasks for the remaining 23 weeks of the project. To keep the timeline brief, requirements have been grouped as their "Features", but there may be overlaps in practice. Weeks in this table are relative to the submission week of the Interim Report. The expected points for the first 16 weeks have also been provided. This has been kept close to around 100 points per week, equivalent to 6 days of work per week.

Week #	Start Date	End Date	Tasks	Points
1	24/11/25	30/11/25	• [V0] - Core • [V0] - "Track" Representation	63 (8 + 55)
2	01/12/25	07/12/25	• [V0] - "Track" Viewing & Selection • [V0] - "Race" Gameplay	128 (39 + 89)
3	08/12/25	14/12/25	• [V0] - "App Settings" • Full V0 testing before V1 • [V1] - "Track" Building - Path	(84) 8 + 76
4	15/12/25	21/12/25	• [V1] - "Track" Building - "Materials"	89
5	22/12/25	28/12/25	• [V1] - "Track" Building - "Obstacles"	89

6	29/12/25	04/01/26	• [V1] - “Track" Building - “Decorations"	89
7	05/01/26	11/01/26	• [V1] - “Track" Management	89
8	12/01/26	18/01/26	• [V1] - “Track" Building - Controls	110
9	19/01/26	25/01/26	• [V1] - “Track" Viewing & Selection	81
10	26/01/26	01/02/26	• [V1] - "Race Car" Selection • [V1] - Extended “Race" Gameplay	149 (60 + 89)
11	02/02/26	08/02/26	• [V1] - Live "Race Details" • Full V1 testing before V2 • [V2] - "Race Options"	102 (34 + 68)
12	09/02/26	15/02/26	• [V2] - "AI Racers"	136
13	16/02/26	22/02/26	• [V2] - "Ghost" Racing • [V2] - "Local Multiplayer" • Full V2 testing before V3 • Preparing V0 – V2 progress for interview	97 (63 + 34)
14	23/02/26	01/03/26	• [V3] - "Track" Sharing	89
15	02/03/26	08/03/26	• [V3] - "Track Leaderboards"	68
16	09/03/26	15/03/26	• [V3] - "Lobby" Management	123
17	16/03/26	22/03/26	• [V3] - "Online Multiplayer" • [V3] - "Race Car Customisation"	97 (55 + 42)
18	23/03/26	29/03/26	Final validation for all requirements	
19	30/03/26	05/04/26	Writing the abstract, introduction, and information sources	
20	06/04/26	12/04/26	Writing the design and implementation	
21	13/04/26	19/04/26	Writing the results and critical appraisal	
22	20/04/26	26/04/26	Proof-reading the document	
23	27/04/26	03/05/26	Dissertation and software due **12pm 27/04/26**! Summarising work for the mini viva	

7.3. Future Timeline Gantt Charts

Gantt charts have been provided below to visualise the timeline with more precise dates.

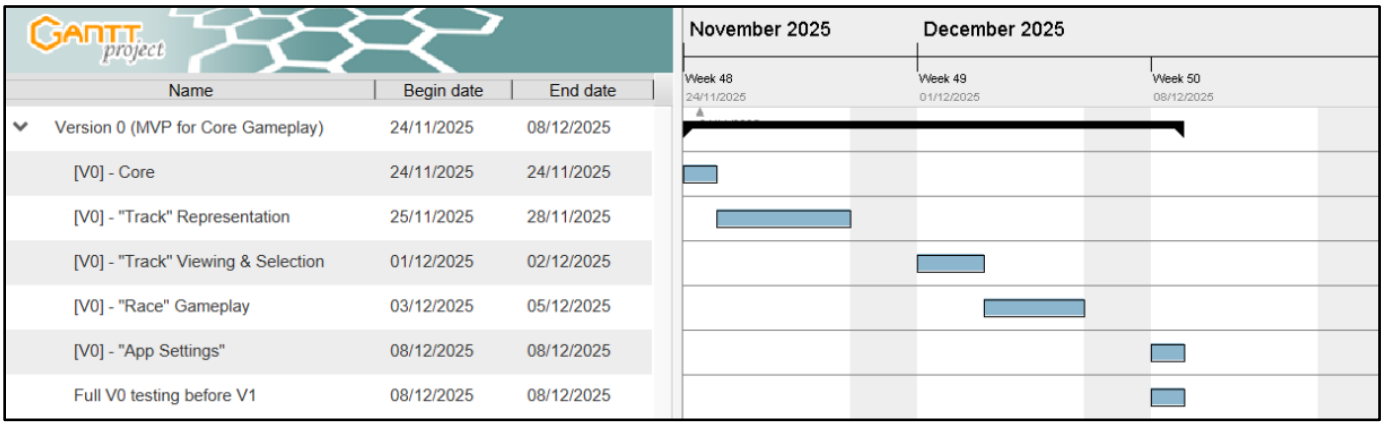


Figure 17 – a timeline of Version 0 development.

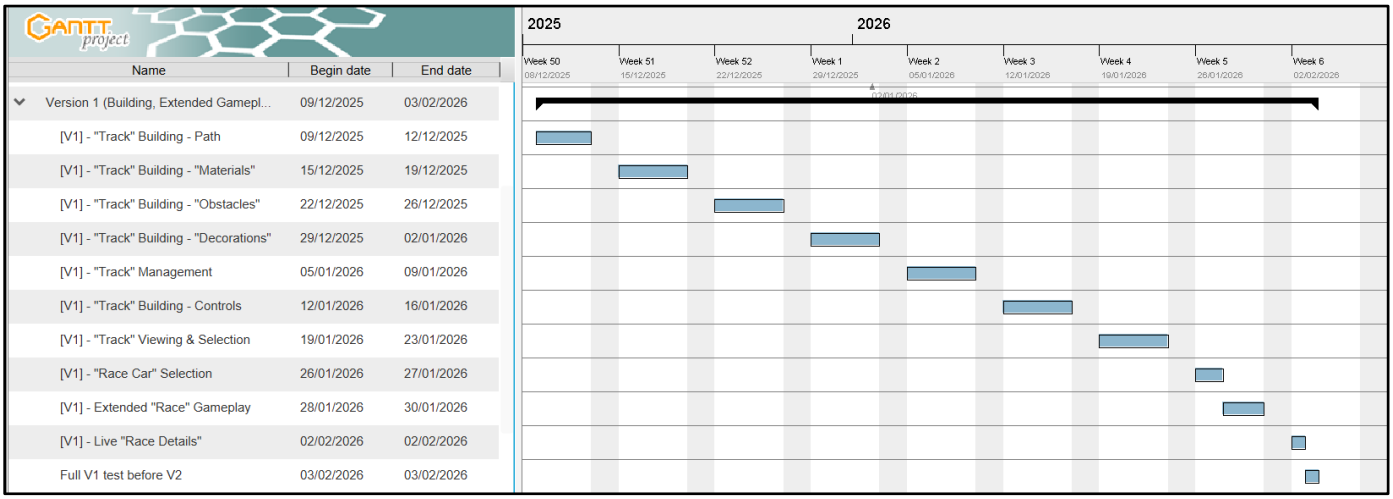


Figure 18 – a timeline of Version 1 development.

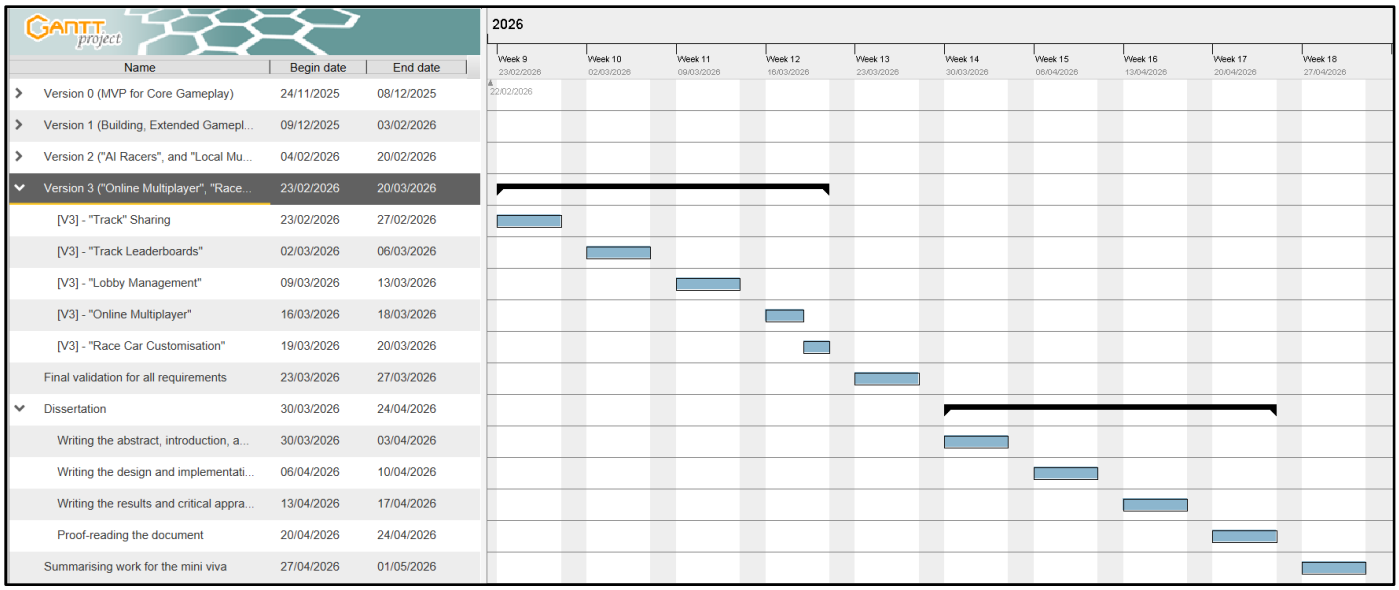


Figure 19 – a timeline of Version 3 development and post-development tasks.

7.4. External Deadlines

During the project, there may be external deadlines which impact its progress for other module assignments. This has been captured in the table below for the first semester. Weeks are relative to the submission week of the Interim Report.

Week #	Date	Assignments
1	20/11/25	Interim Report
2	28/11/25	CO4219 Internet & Cloud Computing – Class Test
4	09/11/25	CO4219 Internet & Cloud Computing – Group Project
4	12/12/25	CO4210 Personal & Group Skills – Individual Presentation
12	07/01/26	CO4217 Agile Cloud Automation – Group Project 2

A daily schedule may need to be set to ensure the project receives adequate work time.

8. References

[1] Martin Gardner, (1973), “Mathematical Games”, Scientific American Magazine Archives, scientificamerican.com, [Online]. Available at: http://www.scientificamerican.com/article/mathematical-games-1973-01/.

[2] J. Russo, L. A. Braggand, and T. Russo, “How primary teachers use games to support their teaching of mathematics”. Deakin University, 01-Mar-2021 [Online]. Available: https://hdl.handle.net/10536/DRO/DU:30150385.

[3] José Pablo Zagal, Miguel Nussbaum and Ricardo Rosas, “A Model to Support the Design of Multiplayer Games”, Presence: Teleoperators and Virtual Environments, 2000, vol. 9, no. 5, pp. 448–462. doi: https://doi.org/10.1162/105474600566943.

[4] M. M. Lehman and J. F. Ramil, “Rules and Tools for Software Evolution Planning and Management”, Annals of Software Engineering, vol. 11, no. 1, pp. 15–44, Nov. 2001.

[5] T. Laszewski and P. Nauduri, “Chapter 8 - Migrating Applications to the Cloud”, in *Migrating to the Cloud*, T. Laszewski and P. Nauduri, Eds Boston: Syngress, 2012, pp. 181–208.

[6] “Inertia Racetrack”, Transum, transum.org, [Online]. Available at: https://www.transum.org/Maths/Game/Vectors/.

[7] Harm Boschloo, “Vector Racer”, Harm Made, harmmade.com, [Online]. Available at: https://harmmade.com/vectorracer/.

[8] “PolyTrack”, Kodub, kodub.com, [Online]. Available at: https://www.kodub.com/apps/polytrack.

[9] P. Massoudi and A. H. Fassihi, ‘Achieving dynamic AI difficulty by using reinforcement learning and fuzzy logic skill metering’, in *2013 IEEE International Games Innovation Conference (IGIC)*, 2013, pp. 163–168.

[10] Dan Dicere, “Artificial Difficulty in Games”, Indiecator, indiecator.org, [Online]. Available at: https://indiecator.org/2024/08/20/artificial-difficulty-in-games/.

[11] Lazzaro, Nicole. "Why we play: affect and the fun of games." in *Human-computer interaction: Designing for diverse users and domains* 155.679-700 (2009): 144.

[12] “GPixel”, JesterDay Studio, jesterday.studio, [Online]. Available at: https://jesterday.studio/en/.

[13] “Formulky”, hraj.si, [Online]. Available at: https://hraj.si/formulky/.

[14] F. Zammetti, “Modern Full-Stack Development”, Apress, 2022, [Online]. doi: 10.1007/978-1-4842-8811-5.

[15] “Phaser – A fast, fun and free open source HTML5 game framework”, Phaser, phaser.io, [Online]. Available at: https://phaser.io/.

[16] “Kaboom.js”, Kaboom.js, kaboomjs.com, [Online]. Available at: https://kaboomjs.com/.

[17] “p5.js”, p5js.org, [Online]. Available at: https://p5js.org/.

[18] “React”, react.dev, [Online]. Available at: https://react.dev/.

[19] “Node.js – Run JavaScript Everywhere”, Node.js, nodejs.org, [Online]. Available at: https://nodejs.org/.

[20] “PHP”, php.net, [Online]. Available at: https://www.php.net/.

[21] “Firebase | Google’s Mobile and Web App Development Platform”, Firebase, firebase.google.com, [Online]. Available at: https://firebase.google.com/.

[22] “What is NoSQL?”, MongoDB, mongodb.com, [Online]. Available at: https://www.mongodb.com/resources/basics/databases/nosql-explained.

[23] “MySQL”, mysql.com, [Online]. Available at: https://www.mysql.com/.

[24] “MongoDB: The World’s Leading Modern Database | MongoDB”, MongoDB, mongodb.com, [Online]. Available at: https://www.mongodb.com/.

[25] “Firebase Hosting”, Firebase, firebase.google.com, [Online]. Available at: https://firebase.google.com/docs/hosting.

[26] “Socket.IO”, socket.io, [Online]. Available at: https://socket.io/.

[27] S. Agarwal and J. R. Lorch, “Matchmaking for online games and other latency-sensitive P2P systems”, in *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication*, Barcelona, Spain, 2009, pp. 315–326.

[28] “Unity Real-Time Development Platform | 3D, 2D, VR & AR Engine”, Unity, unity.com, [Online]. Available at: https://unity.com/.

[29] J. Gregory, (2018). Game Engine Architecture (3rd ed.). A K Peters/CRC Press. https://doi.org/10.1201/9781315267845.

[30] “Decoupling Patterns”, Game Programming Patterns, gameprogrammingpatterns.com, [Online]. Available at: https://gameprogrammingpatterns.com/decoupling-patterns.html.

[31] “Azure DevOps | Microsoft Azure”, Microsoft Azure, azure.microsoft.com, [Online]. Available at: https://azure.microsoft.com/en-us/products/devops.

[32] T. Kravchenko, T. Bogdanova, and T. Shevgunov, “Ranking Requirements Using MoSCoW Methodology in Practice”, in *Cybernetics Perspectives in Systems*, 2022, pp. 188–199.

[33] “Firebase”, Devopedia, devopedia.org, [Online]. Available at: https://devopedia.org/firebase.

[34] B. Ozdemir and O. Caliskan, “Menu Design: A Review of Literature”, *Journal of Foodservice Business Research*, vol. 18, no. 3, pp. 189–206, May 2015.

[35] javidx9, “Programming Racing Lines”, YouTube, youtube.com, [Online]. Available at: https://www.youtube.com/watch?v=FlieT66N9OM.

[36] A. Reshetov, A. Soupikov, and J. Hurley, “Multi-level ray tracing algorithm”, *ACM Trans. Graph.*, vol. 24, no. 3, pp. 1176–1185, July 2005.

[37] Chen, Jing-Chao. "Dijkstra’s shortest path algorithm." *Journal of formalized mathematics* 15.9 (2003): 237-247.

[38] M. L. Fredman and R. E. Tarjan, “Fibonacci Heaps And Their Uses In Improved Network Optimization Algorithms”, in *25th Annual Symposium on Foundations of Computer Science, 1984.*, 1984, pp. 338–346.

[39] K. J. Lieberherr and I. M. Holland, “Assuring good style for object-oriented programs”, *IEEE Software*, vol. 6, no. 5, pp. 38–48, 1989.

[40] Jenna Jonsson, “Game Development: Using UML Class Diagram”, [Online]. Available at: https://www.diva-portal.org/smash/get/diva2:631806/FULLTEXT01.pdf.

[41] “Vitest”, vitest.dev, [Online]. Available at: https://vitest.dev/.

[42] Muhammad Mogra, “Project Log”, GitLab.com, [Online]. Available at: https://campus.cs.le.ac.uk/gitlab/ug_project/25-26/mm994/-/blob/main/PROJECTLOG.md.

[43] G. Waja, J. Shah, and P. Nanavati, “AGILE SOFTWARE DEVELOPMENT”, *International Journal of Engineering Applied Sciences and Technology*, vol. 5, Apr. 2021.

[44] “Define features and epics to organize your backlog”, Microsoft Learn, learn.microsoft.org, [Online]. Available at: https://learn.microsoft.com/en-us/azure/devops/boards/backlogs/define-features-epics?view=azure-devops&tabs=agile-process.

[45] Muhammad Mogra, “CS Project II Team Project Backlog – Boards”, Microsoft Azure, dev.azure.com, [Online]. Available at: https://dev.azure.com/MuhammadsOrganisation/CS%20Project%20II/_backlogs/backlog/